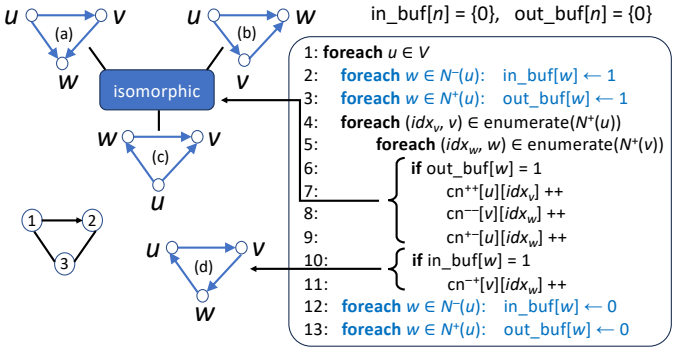


Fig. 8. Initial Counter Computation

Appendix A
Proofs of Theorems 4–6

Proof of Theorem 4. Since P is a (k_1, k_2)-plex, $|\overline{N_P^+}(u)| \leq k_1$ and $|\overline{N_P^+}(v)| \leq k_1$, where both sets include the vertex itself (i.e., $u \in N_P^+(u)$ and $v \in \overline{N_P^+}(v)$). Any vertex $w \in P$ either belongs to $N_P^+(u) \cap N_P^+(v)$, or belongs to $\overline{N_P^+}(u) \cup \overline{N_P^+}(v)$. Thus,

$$\begin{aligned}|P| &= |N_P^+(u) \cap N_P^+(v)| + |\overline{N_P^+}(u) \cup \overline{N_P^+}(v)| \\ &= |N_P^+(u) \cap N_P^+(v)| + |\overline{N_P^+}(u)| + |\overline{N_P^+}(v)| \\ &\quad - |\overline{N_P^+}(u) \cap \overline{N_P^+}(v)| \\ &\leq |N_P^+(u) \cap N_P^+(v)| + 2k_1 - |\overline{N_P^+}(u) \cap \overline{N_P^+}(v)|.\end{aligned}$$

Now, $u \in \overline{N_P^+}(v)$ iff $(v, u) \notin E$, and $v \in \overline{N_P^+}(u)$ iff $(u, v) \notin E$. Hence $\overline{N_P^+}(u) \cap \overline{N_P^+}(v)$ contains at least δ vertices among u and v, where δ is defined in Theorem 4, so $|N_P^+(u) \cap N_P^+(v)| \geq |P| - 2k_1 + \delta \geq q - 2k_1 + \delta$. □

Proof of Theorem 5. Since $N_P^-(u) \cap N_P^+(v) = N_P^+(v) \cap N_P^-(u)$, Theorem 5 is exactly Theorem 3 applied to the ordered pair (v, u), so the condition becomes whether $(v, u) \in E$. □

Proof of Theorem 6. Reversing the direction of every edge turns a (k_1, k_2)-plex into a (k_2, k_1)-plex and swaps in- and out-neighbors, which turns Theorem 4 into Theorem 6. □

Appendix B
Counter Computation

Initial Counter Computation. Figure 8 shows how we compute the initial values of the counters. Specifically, consider an arbitrary (starting) edge (v_1, v_2), then we can obtain a triangle $\Delta v_1 v_2 v_3$ with four possible edge orientations (a)–(d) as shown on the left of Figure 8. Note that the three acyclic configurations are isomorphic: each triangle of type (b) (resp. (c)) has a one-to-one mapping to a triangle of type (a), so we just need to enumerate triangles of type (a) to avoid redundancy.

Figure 8 shows this enumeration where $v \in N^+(u)$ (Line 4) and $w \in N^+(v)$ (Line 5), and Line 6 checks if $w \in N^+(u)$. Note that for each vertex u, we save its counters, such as $cn^{++}(u, :)$, also sparsely to be space-efficient. For each position idx in the adjacency list $N^+(u)$ such that $N^+(u)[idx] = v$, we save $cn^{++}[u][idx] = cn^{++}(u, v)$.

If Line 6 finds that $w \in N^+(u)$, we increment the corresponding counters for the three edges in the triangle, i.e., $cn^{++}(u, v)$, $cn^{--}(v, w)$, and $cn^{+-}(u, w)$. To avoid repeated binary search when checking if $w \in N^+(u)$ in Line 6, we use a bitmap out_bf of size $|V|$ that is initialized as all zeros. When we process a vertex $u \in V$ in Line 1, we first set the bits of all the out-neighbors in $N^+(u)$ to be 1 in Line 3, so that Line 6 can be implemented as a simple bit check. These bits are cleared in Line 13 before we continue to process the next vertex $u \in V$ in Line 1.

The cyclic triangle of type (d) increments cn^{-+}. In Line 10, if we find $w \in N^-(u)$, we increment $cn^{-+}(v, w)$ only since $cn^{-+}(w, u)$ (resp. $cn^{-+}(u, v)$) will be incremented when we enumerate from starting vertex v (resp. w) in Line 1. □

Incremental Counter Maintenance. Let us consider edge peeling first. When we remove an edge (u, v), (1) we decrement $d^+(u)$ and $d^-(v)$. (2) For each $w \in N^+(u) \cap N^-(v)$, we decrement $cn^{++}(u, w)$ and $cn^{--}(w, v)$ (see Figure 3(b) to get the intuition). Similarly, (3) for each $w \in N^+(u) \cap N^+(v)$, we decrement $cn^{+-}(u, w)$ and $cn^{--}(v, w)$; (4) for each $w \in N^-(u) \cap N^+(v)$, we decrement $cn^{-+}(w, u)$ and $cn^{-+}(v, w)$; (5) for each $w \in N^-(u) \cap N^-(v)$, we decrement $cn^{++}(w, u)$ and $cn^{+-}(w, v)$.

Whenever the decrement of a common-neighbor counter for an edge e causes its value to be below the threshold stipulated by Theorems 3–6, e is added to an edge queue Q_e for further peeling. Vertices whose degrees get decremented below the thresholds stipulated by Theorem 2 are also added to a vertex queue Q_v. When edge peeling completes (i.e., $Q_e = \emptyset$), we conduct vertex peeling starting from the vertices in Q_v. The removal of each vertex w in Q_v will also incrementally decrement the degrees (resp. common-neighbor counters) of w's neighbors (resp. neighbor-pairs), for which we omit the details. Once vertex peeling completes, new edges may have been added to Q_e so we start another round of edge peeling. We alternate between edge peeling and vertex peeling until both Q_e and Q_v are empty. This process is similar to the CTCP algorithm (Algorithm 3) of [11] in the k-plex context. □

Note that the above process of shrinking G by alternating edge peeling and vertex peeling is executed on top of the $(q - k_1, q - k_2)$-core of G which is initially computed by a vertex peeling algorithm, since the latter is more efficient in pruning unpromising vertices as a preprocessing.

Appendix C
Proof of Theorem 7

Proof. Consider any two vertices u, v in a (k_1, k_2)-plex P with $|P| \geq q$ and $k_1, k_2 \leq \frac{q+1}{2}$. We show that u and v cannot be more than 2 hops apart, i.e., cannot fall out of the 6 cases in Figure 9.

Without loss of generality, we only consider the path from v to u where the first edge is outbound from v, i.e., Cases 1(a)–(c). Cases 2(a)–(c) are symmetric and can be similarly proved.

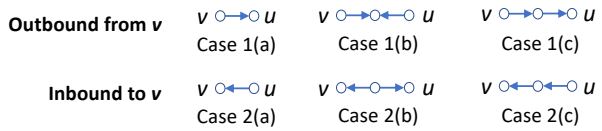


Fig. 9. Cases for Two-Hop Diameter Upper Bound

If v directly points to u, we are done since Case 1(a) occurs. Now assume that edge $(v,u) \notin E$, and we show that:

- **Case (I): $(\mathbf{u},\mathbf{v}) \notin \mathbf{E}$,** then both Case 1(b) and Case 1(c) should be satisfied. **(i) We first prove Case 1(b).** Note that $u \notin N^+(v)$ and $v \notin N^+(u)$. Since $k_1 \le \frac{q+1}{2} \le \frac{|P|+1}{2}$, u and v each points to at least $|P| - k_1 \ge \frac{|P|-1}{2}$ other vertices in P, so they must share an out-neighbor; otherwise, there exist $2 \cdot \frac{|P|-1}{2} = |P| - 1$ vertices other than u and v, leading to a contradiction since there will be at least $(|P|+1)$ vertices in P when adding u and v. **(ii) We next prove Case 1(c).** Note that $v \notin N^-(u)$ and $u \notin N^+(v)$. Since $k_1 \le \frac{q+1}{2} \le \frac{|P|+1}{2}$, v points to at least $|P| - k_1 \ge \frac{|P|-1}{2}$ other vertices in P; also since $k_2 \le \frac{q+1}{2} \le \frac{|P|+1}{2}$, u is pointed to by at least $|P| - k_2 \ge \frac{|P|-1}{2}$ other vertices in P. So, $N^+(v)$ and $N^-(u)$ must intersect as illustrated by Figure 9 Case 1(c); otherwise, there will be $(|P|+1)$ vertices in P when adding u and v.
- **Case (II): $(\mathbf{u},\mathbf{v}) \in \mathbf{E}$,** then Case 1(c) should be satisfied. The proof is the same as (ii) above. Note that we cannot guarantee Case 1(b) anymore, since $v \in N^+(u)$, i.e., v can be one of the at least $|P| - k_1 \ge \frac{|P|-1}{2}$ neighbors of u, invalidating the proof for (i) above.

Symmetrically, consider the path from v to u where the first edge is inbound to v, i.e., Cases 2(a)–(c). If u directly points to v, we are done since Case 2(a) occurs. If edge $(u,v) \notin E$:

- **Case (III): edge (v,u) does not exist in G,** then both Case 2(b) and Case 2(c) should be satisfied. The proof is symmetric to Case (I) above and thus omitted.
- **Case (IV): edge (v,u) exists in G,** then Case 2(c) should be satisfied. The proof is symmetric to Case (II) above.

Putting the above analysis together, we obtain the following 4 cases, for each of which we explain how to prune an impossible candidate $u \in ext(S)$ given a vertex $v \in S$, where $ext(S)$ denotes the set of candidate vertices that can extend S into a valid (k_1,k_2)-plex.

- **Case A: $(v,u) \in E$ and $(u,v) \in E$.** In this case, we always have $u \in ext(S)$.
- **Case B: $(v,u) \notin E$ and $(u,v) \in E$.** Based on Case (II) above, we have $u \in ext(S)$ only if a path $u \leftarrow w \leftarrow v$ exists in G for some $w \in P - \{u,v\}$.
- **Case C: $(v,u) \in E$ and $(u,v) \notin E$.** Based on Case (IV) above, we have $u \in ext(S)$ only if a path $u \rightarrow w \rightarrow v$ exists in G for some $w \in P - \{u,v\}$.
- **Case D: $(v,u) \notin E$ and $(u,v) \notin E$.** Based on Case (I) above, we have Condition (C1): $u \in ext(S)$ only if both Case 1(b) and Case 1(c) in Figure 9 are satisfied. Similarly, based on Case (III) above, we have Condition (C2): $u \in ext(S)$ only if both Case 2(b) and Case 2(c) are satisfied. Combining both conditions, $u \in ext(S)$ only if there exist $w_1, w_2, w_3, w_4 \in P - \{u,v\}$ such that $u \leftarrow w_1 \leftarrow v$ and $u \leftarrow w_2 \rightarrow v$ and $u \rightarrow w_3 \leftarrow v$ and $u \rightarrow w_4 \rightarrow v$.

Based on the above analysis, Algorithm 2 computes the set of candidates u that are not 2-hop pruned by v. Line 1 keeps all bidirectional neighbors $D = N^{\pm}(v)$, which always belong to $\mathrm{ext}(S)$ by Case A. Line 2 handles Case C: an out-only neighbor o (i.e., $(v,o) \in E$ and $(o,v) \notin E$) is kept only if there exists a path $o \rightarrow w \rightarrow v$, i.e., $N^+(o) \cap N^-(v) \neq \emptyset$. Symmetrically, Line 3 handles Case B: an in-only neighbor i is kept only if there exists a path $v \rightarrow w \rightarrow i$, i.e., $N^-(i) \cap N^+(v) \neq \emptyset$. Let $S_O = D \cup O'$ and $S_I = D \cup I'$ denote the kept out- and in-neighbors of v, respectively. Line 5 handles Case D: a vertex u non-adjacent to v is kept only if it has both an in-neighbor and an out-neighbor in S_O (i.e., paths $u \leftarrow w_1 \leftarrow v$ and $u \rightarrow w_3 \leftarrow v$), as well as both an in-neighbor and an out-neighbor in S_I (i.e., paths $u \leftarrow w_2 \rightarrow v$ and $u \rightarrow w_4 \rightarrow v$). Since $w_1, \ldots, w_4 \in P$ are adjacent to v, they are themselves kept by Lines 2–3, so it is safe to use the pruned S_O and S_I in Line 5. Finally, the four candidate sets corresponding to Cases A–D are unioned and returned as $\mathbb{B}(v)$.

One may wonder whether Lines 2–3 should be re-applied after O' and I' are computed, i.e., whether removing some in-only neighbor i could invalidate the path $o \rightarrow i \rightarrow v$ that kept an out-only neighbor o. This cannot happen: if o is kept due to an edge $o \rightarrow i$ with i an in-only neighbor, then the path $v \rightarrow o \rightarrow i$ also satisfies the condition of Line 3 for i, so i is kept as well; the case for in-only neighbors is symmetric. Hence O' and I' already form a fixed point, and re-applying these lines removes no further vertices. □

Appendix D
Complexity Analysis.

Given a directed graph $G = (V,E)$, we evaluate the worst-case running time of Algorithm 4. Let $T(m,n)$ be the worst-case running time of function BK2$(G, k_1, k_2, q, P, C, X)$ where $m = |C|$ and $n = |P \cup C|$, and we define $k = max(k_1,k_2)$. Next, we analyze $T(m,n)$ by considering four different cases:

I. When $C = \emptyset$ (Lines 4–6), then

$$T(m,n) = T(0,n) \le c_1, \tag{4}$$

where c_1 is a constant.

II. When $P \cup C$ is not a (k_1,k_2)-plex and the pivot vertex is in P (Lines 8–17). Here, $p+1$ branches are created each with a smaller candidate set C'. Before each call the sets C' and X' are further shrunk in Lines 15–16 which takes time $O(|V|^2)$. The worst-case running time of this case is, therefore

$$T(m,n) \le (p+1)c_2|V|^2 + \sum_{i=1}^{p} T(m-i, n-1) + T(m-d, n-d+p), \tag{5}$$

where c_2 is a constant.

III. When $P \cup C$ is a (k_1, k_2)-plex (Lines 19–24), then no further branches are created, and the cost in this case is only incurred by updating X' in Line 22, which is

$$T(m, n) \leq c_2|V|^2 \tag{6}$$

IV. When $P \cup C$ is not a (k_1, k_2)-plex and the pivot vertex is in C (Lines 25–30). The cost in this case is

$$T(m, n) \leq c_2|V|^2 + T(m-1, n) + T(m-1, n-1).$$

In the worst case, no candidate vertex is reduced from $C - \{v_p\}$ in Line 28 so v_p remains the minimum degree vertex in the subsequent recursive call at Line 30 (within which $v_p \in P' = P \cup \{v_p\}$). Hence, by expanding $T(m-1, n)$ using Equation (5) we have:

$$\begin{aligned} T(m, n) \leq (p+2)c_2|V|^2 + \sum_{i=1}^{p} T(m-1-i, n-1) \\ + T(m-1-d, n-d+p) + T(m-1, n-1). \end{aligned} \tag{7}$$

Next, we compute a closed-form upper-bound of $T(m, n)$ which satisfies the Equations (4)–(7), and we can drop the argument n from $T(m, n)$ since the base case expressions in Equations (4) and (6) are not a function of n. Then, the largest $T(m)$ is upper-bounded by Equations (4) and (7), i.e.

$$T(m) \leq \begin{cases} c_1 & \text{if } m = 0 \\ (p+2)c_2|V|^2 + \sum_{i=0}^{p} T(m-1-i) & \\ + T(m-1-d) & \text{otherwise} \end{cases} \tag{8}$$

Since $p \leq d-1$ and $p < k$, the worst-case is reached when $p = k-1$ and $d = k$. Then, Equation (8) can be rewritten as:

$$T(m) \leq \begin{cases} c_1 & \text{if } m = 0 \\ (k+1)c_2|V|^2 + \sum_{i=0}^{k} T(m-1-i) & \text{otherwise} \end{cases} \tag{9}$$

We show that $T(m) = \mathcal{O}(\alpha_k \gamma_k^m - \beta_k)$ where γ_k is largest real root of $x^{k+2} - 2x^{k+1} + 1 = 0, \beta_k = \frac{(k+1)c_2|V|^2}{k}$ and $\alpha_k = \frac{(2^k k+1)\beta_k + 2^k c_1}{\gamma_k^{k+1}}$.

Lemma 1. *Let $H(m) = \alpha_k \gamma_k^m - \beta_k$ where γ_k is largest real root of $x^{k+2} - 2x^{k+1} + 1 = 0, \beta_k = \frac{(k+1)c_2|V|^2}{k}$ and $\alpha_k = \frac{(2^k k+1)\beta_k + 2^k c_1}{\gamma_k^{k+1}}$. If $T(m)$ satisfies Equation (9), then $T(m) \leq H(m)$ for $m \geq 0$.*

Proof. We first show $T(m) \leq 2^{m-1} k\beta_k + 2^{m-1} c_1$ by induction on m.

Basis: for $m = 1, T(1) \leq (k+1)c_2|V|^2 + T(0) \leq k\beta_k + c_1$.

Induction: We assume that $T(i) \leq 2^{i-1} k\beta_k + 2^{i-1} c_1$ for all $i = 1, \ldots, m-1$ and prove that it also holds for $i = m$. By Equation (9) we have:

$$\begin{aligned} T(m) &\leq (k+1)c_2|V|^2 + T(m-1) + \cdots + T(0) \\ &= (k+1)c_2|V|^2 + c_1 + \sum_{i=1}^{m-1} T(i) \\ &\leq (k+1)c_2|V|^2 + c_1 + \sum_{i=1}^{m-1} (2^{i-1} k\beta_k + 2^{i-1} c_1) \\ &= k\beta_k (1 + \sum_{i=1}^{m-1} 2^{i-1}) + c_1 (\sum_{i=1}^{m-1} 2^{i-1} + 1) \\ &= 2^{m-1} k\beta_k + 2^{m-1} c_1 \end{aligned} \tag{10}$$

Then, we prove that $T(m) \leq H(m)$ for all $m \geq 0$ again by induction on m.

Basis: for $m = 1, T(1) \leq (k+1)c_2|V|^2 + T(0) \leq k\beta_k + c_1$, and $H(1) = \alpha_k \gamma_k - \beta_k = \frac{(2^k k+1)\beta_k + 2^k c_1}{\gamma_k^k} \geq k\beta_k + c_1$ (since $\gamma_k < 2$ and hence $\gamma_k^k < 2^k$).

Induction: We assume that $T(m) \leq H(m)$ for $m = 1, \ldots, i$ and prove that it also holds for $m = i+1$. First, let us factorize $x^{k+2} - 2x^{k+1} + 1$ as $(x-1)(x^{k+1} - x^k - \cdots - 1)$. Denote $F_k(x) = (x^{k+1} - x^k - \cdots - 1)$. We have $F_k(1) < 0$ and $F_k(2) > 0$, indicating that $F_k(x) = 0$ has a root between $(1, 2)$. Combining the fact that γ_k is also the root of $F_k(x) = 0$, thus $\gamma_k^k + \cdots + 1 = \gamma_k^{k+1}$. Multiply both side with γ_k^{i-k}, we have $\gamma_k^i + \cdots + \gamma_k^{i-k} = \gamma_k^{i+1}$. Then the following inequalities hold.

$$\begin{aligned} T(i+1) &\leq T(i) + \cdots + T(i-k) + (k+1)c_2|V|^2 \\ &\leq \alpha_k(\gamma_k^i + \cdots + \gamma_k^{i-k}) - (k+1)\beta_k + k\beta_k \\ &\leq \alpha_k(\gamma_k^i + \cdots + \gamma_k^{i-k}) - \beta_k \\ &= \alpha_k \gamma_k^{i+1} - \beta_k \\ &= H(i+1) \end{aligned} \tag{11}$$

□

Finally, as we call BK2(.) with $m = |V|$ and k is a constant, we get the worst-case running time of our algorithm as given in Theorem 8.

Appendix E
Incremental Set and Degree Maintenance

Global Container Design. Instead of creating new sets P', C' and X' as the inputs into each recursive call BK2($G, k_1, k_2, q, P', C', X'$) (see Lines 17, 27 and 30), we propose to reuse the same containers for P, C and X throughout the recursive execution, by initializing their spaces at the very beginning, and populating them with proper elements for use in each recursion body.

In particular, we maintain three global containers for P, C and X, respectively, using the data structure shown in Figure 10. Specifically, two arrays with capacity $|V|$ are maintained for each set S: (1) *list*[] which keeps the list of S's elements for scanning and adding new elements; (2) *pos*[]

TABLE V

THE ENUMERATION TIME (SEC) OF MINING MAXIMAL (k_1, k_2)-PLEXES IN PARALLEL EXECUTION ON ALL DATASETS. TIMES EXCLUDE THE ONE-TIME SERIAL PREPROCESSING (CORE AND SECOND-ORDER PRUNING OF THE INPUT GRAPH).

Dataset	k_1	k_2	q	# of (k_1, k_2)-plexes	1 Thread	2 Threads	4 Threads	8 Threads	16 Threads	32 Threads
bitcoin	2	5	10	144	0.08	0.04	0.02	0.01	0.01	0.01
			12	0	0.01	0.00	0.00	0.00	0.00	0.00
	3	5	10	24,237	10.58	5.41	2.71	1.34	0.67	0.34
			12	286	1.32	0.68	0.34	0.17	0.09	0.05
wiki-vote	3	5	10	471	2.31	1.15	0.57	0.29	0.14	0.07
			12	0	0.10	0.05	0.03	0.01	0.01	0.00
	4	5	10	30,109	430.88	219.47	109.94	54.35	27.02	13.57
			12	93	20.40	10.45	5.24	2.57	1.30	0.65
mathoverflow	2	2	10	523,633	38.70	19.76	9.82	4.95	2.47	1.27
			12	150,888	18.18	9.28	4.62	2.33	1.17	0.64
	2	3	10	1,762,917	97.56	50.51	24.59	12.37	6.23	3.24
			12	602,862	49.59	25.21	12.53	6.26	3.21	1.85
as-caida	2	3	10	23,314	0.64	0.33	0.16	0.09	0.05	0.05
			15	185	0.03	0.01	0.01	0.00	0.00	0.00
	3	4	15	17,303	1.20	0.62	0.31	0.16	0.08	0.07
			20	0	0.01	0.01	0.00	0.00	0.00	0.00
epinions	2	3	10	773,095	37.84	19.25	9.52	4.75	2.46	1.40
			12	252,511	16.02	8.27	4.11	2.09	1.06	0.72
	3	4	12	24,588,560	2664.41	1349.59	672.88	337.83	167.22	84.41
			15	2,380,874	510.90	252.75	126.47	62.78	31.38	15.89
email-euall	2	3	10	17,438	2.44	1.26	0.63	0.32	0.16	0.12
			12	1442	0.75	0.38	0.19	0.10	0.05	0.04
	3	4	12	196,806	103.78	52.43	26.06	13.03	6.50	3.25
			15	1351	11.38	5.80	2.87	1.45	0.72	0.37
amazon0505	3	5	10	4696	0.09	0.05	0.02	0.01	0.01	0.00
			12	24	0.01	0.00	0.00	0.00	0.00	0.00
	5	3	10	1598	0.04	0.02	0.01	0.01	0.00	0.00
			12	1	0.00	0.00	0.00	0.00	0.00	0.00
web-google	2	3	15	997	0.06	0.03	0.01	0.01	0.01	0.00
			20	57	0.01	0.01	0.00	0.00	0.00	0.00
	3	4	15	1993	0.08	0.04	0.02	0.01	0.01	0.01
			20	61	0.02	0.01	0.01	0.00	0.00	0.00
soc-pokec	2	3	15	17,980	1.59	0.83	0.41	0.21	0.11	0.12
			20	17	0.04	0.02	0.01	0.01	0.00	0.01
	3	3	15	508,884	66.64	33.88	16.76	8.43	4.20	2.12
			20	648	0.96	0.50	0.24	0.13	0.07	0.04
wiki-talk	2	2	15	22,219	373.75	191.12	95.75	47.90	23.82	11.91
			20	0	88.34	45.15	22.44	11.35	5.61	2.84
	2	3	15	165,156	799.96	412.59	203.83	102.63	51.60	25.98
			20	0	176.28	91.85	45.37	22.77	11.40	5.71
arabic-2005	2	2	1000	106,895	3998.38	2055.73	1049.55	657.99	499.63	252.24
	3	3	3000	3500	116.70	59.58	29.89	19.79	16.64	10.53
uk-2005	2	2	200	118,547	454.14	228.08	114.88	57.83	28.38	14.40
	3	3	400	69,815	223.60	114.16	56.65	28.41	14.11	7.16
it-2004	2	2	1000	15,087	1300.20	664.24	334.96	177.33	111.19	90.18
	3	3	3000	1034	82.50	42.21	21.38	12.19	7.86	5.82
webbase-2001	2	2	300	2,884,394	1793.17	919.18	481.06	242.50	128.21	86.74
	3	3	500	88,781	284.77	147.90	76.75	38.21	20.37	10.60
clue-web	2	2	10	1,215,977	18.83	9.80	4.88	2.52	1.43	1.45
	3	3	20	95,383	34.54	17.57	8.71	4.38	2.18	1.15

which maps each vertex ID back to its position in *list*[], to facilitate element search and removal. Note that element addition, search and removal can all be done in $O(1)$ time. To illustrate using Figure 10, $S.add(10)$ appends 10 to *list*[] at position 3, and records *pos*[10] $=$ 3; while $S.remove(3)$ first finds the element position in *list*[] as *pos*[3] $=$ 1, and then erases it in *list*[] by moving the last element 10 to *list*[1], and moving 1 from *pos*[3] to *pos*[10]. In this way, we can incrementally populate $\langle P, C, X \rangle$ for calling BK2(.) with minimal cost.

For example, for the else-branch in Lines 27–30, we can first conduct $C.remove(v_p)$ and $X.add(v_p)$ before calling BK2(.) at

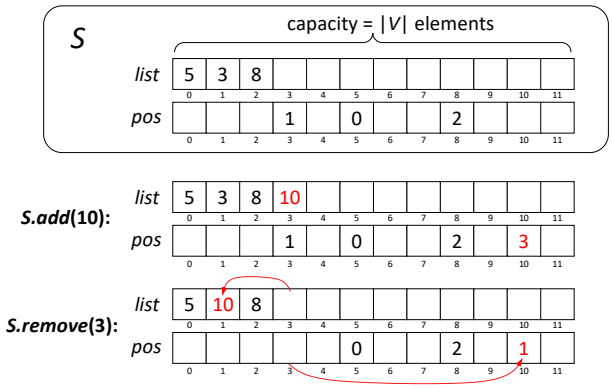


Fig. 10. Data Structure for Set Maintenance

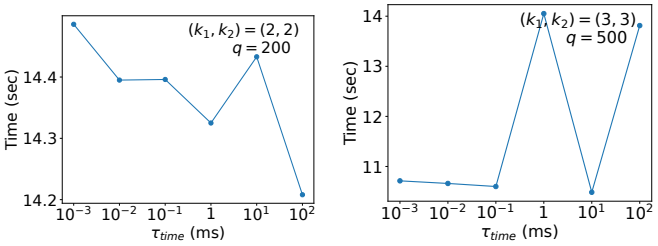
Line 27, and then recover X for use by Line 30 by conducting $X.remove(v_p)$. Line 28 (resp. Line 29) then removes from C (resp. X) those elements v such that $P \cup \{v_p, v\}$ is not a (k_1, k_2)-plex, and let us denote the set of removed elements by C'' (resp. X''). Then, we can conduct $P.add(v_p)$ and call BK2(.) with (P, C, X) in Line 30, after which we recover P, C and X for backtracking by conducting $P.remove(v_p)$, $C.add(C'')$ and $X.add(X'')$. Note that C'' and X'' are kept in global buffers whose spaces are initialized at the beginning, and reused throughout the recursive execution.

Incremental Set and Degree Maintenance. We also incrementally maintain the degree information required by the recursive enumeration. Specifically, we maintain (1) $d^+_{P\cup C}(.)$ and $d^-_{P\cup C}(.)$ which are needed in Lines 8–10 and 19–22; and (2) $d^+_P(.)$ and $d^-_P(.)$ which are needed in Lines 15–16 and 28–29, as we have explained at the end of Section III. For example, whenever we remove a vertex v from C, we need to decrement $d^-_{P\cup C}(u)$ for all $u \in N^+_{P\cup C}(v)$ and $d^+_{P\cup C}(u)$ for all $u \in N^-_{P\cup C}(v)$.

Task Materialization for Parallel Execution. In the parallel algorithm described in Section VII, each task corresponds to a recursive state represented by $\langle P, C, X\rangle$. We now let each thread maintain its own global containers for P, C and X as well as buffers C'' and X'', which are reused when the thread processes its fetched tasks one after another.

A naïve implementation would materialize a task by copying the full internal state of the containers, including both the *list*[] and *pos*[] arrays for each of P, C, and X. However, while *list*[] only stores the active elements, *pos*[] is an auxiliary indexing structure of size $|V|$ used to support $O(1)$ element access and updates. As a result, copying *pos*[] incurs $O(|V|)$ overhead per task, which is prohibitively expensive when a large number of tasks are generated.

To address this, we adopt a compact task representation that stores only the active elements in *list*[] for each set, without materializing the corresponding *pos*[] array. When a task is fetched by a worker thread from the task queue Q for execution, the *pos*[] array is reconstructed locally by scanning the stored elements in *list*[] and assigning their positions accordingly. This reconstruction takes $O(|P|+|C|+|X|)$ time by scanning only the active elements, which is typically much smaller than the $O(|V|)$ cost of copying the full auxiliary index arrays.



(a) uk-2005 (b) webbase-2001

Fig. 11. Running Time When τ_{time} is Varied

For the example shown in Figure 10, we only need to store the three elements 5, 3 and 8 with the created task, the cost of which is negligible. When the task is fetched by a thread for processing, the thread reconstructs the *pos*[] array by assigning $pos[5] = 1$, $pos[3] = 2$, and $pos[8] = 3$ according to their positions in *list*[]. This allows the thread to fully restore the internal state of the container and continue the recursive execution with $O(1)$ support for element addition, removal, and lookup.

This design significantly reduces both memory usage and task creation overhead, while preserving the constant-time set operations required during recursive enumeration.

Appendix F
Complete Results in Parallel Executions

We implemented the parallel version of Ours based on the description in Section VII. Table V reports the (k_1, k_2)-plex enumeration time of parallel Ours, which excludes the one-time serial preprocessing of the input graph, on all our datasets in Table I when running with 1, 2, 4, 8, 16 and 32 threads, respectively, where we use the default timeout threshold $\tau_{\text{time}} = 0.1$ ms.

As Table V shows, our parallel algorithm scales well with the number of CPU cores: for the 23 settings on the small and medium datasets whose 1-thread enumeration takes at least 1 second, 32 threads achieve 13.3×–32.1× speedups. The remaining settings finish within 1 second even with 1 thread, so their speedups are limited by fixed overheads.

Appendix G
Load Balancing

We also conducted experiments to study the impact of τ_{time}. Recall from Section VII that if a task runs for a period of more than τ_{time}, it will decompose itself so that the decomposed tasks can be processed in parallel. Figure 11 shows the enumeration time when using 32 threads and varying τ_{time} as 10^{-3}, 10^{-2}, 10^{-1}, 1, 10, 100 ms for two representative settings on our large graphs uk-2005 and webbase-2001. On uk-2005, the enumeration time varies within 2%, since its many initial tasks already balance the workload. On webbase-2001, $\tau_{\text{time}} \leq 0.1$ ms consistently gives short enumeration times (10.6–10.7 s), while larger values can increase the time by up to 33% (14.05 s with $\tau_{\text{time}} = 1$ ms): a large τ_{time} delays

the decomposition of long-running tasks, which may then become stragglers that keep running while the other threads are already idle. Our default setting $\tau_{\mathrm{time}} = 0.1$ ms is thus among the best settings on both graphs.